LINE WATCH

Real-Time Betting-Line Divergence Detection

Big Data Course Project: Final Report

CSE412 Big Data & Large-Scale Computing

Applied Project, Track D (Kafka → Spark Streaming → Hive/HDFS)

Submitted by

Name	**Roll Number**
Jen Jose Jeeson	2023BCS0017
Rupsika Thipparthi	2023BCS0197
Praneetha Pillai	2023BCS0207

Project repository: github.com/jenjose72/betting-line-divergence

September 2026

ABSTRACT

Betting odds for a football match should mainly change when public information changes, for example after a goal or a red card. Odds that move sharply without anything on the pitch to explain them may point to leaked information or an inefficient market, and are therefore of interest in sports-integrity monitoring. In this project, Line Watch, we built a real-time pipeline that compares betting-line movement with the match events that should explain it. Opening and closing Bet365 odds from football-data.co.uk are interpolated into minute-level ticks, and real event timelines are taken from API-Football. Both are replayed through two Apache Kafka topics, processed by a Spark Structured Streaming job, stored as Parquet on HDFS, exposed as Hive external tables and shown on a Streamlit dashboard. For every five-minute window the job converts odds into normalised implied probabilities and computes divergence_score = actual_move - expected_move, where expected movement comes from fixed, rule-based event priors; no machine learning is used. Two real Premier League matches produced no flagged windows, while a clearly labelled synthetic leak case was flagged in exactly one window (divergence +0.184). The main limitation is that the odds are interpolated, not real tick-level market data.

1. Introduction

Bookmakers publish decimal odds for home win, draw and away win. The inverse of a decimal odd is an implied probability, so odds of 2.0 mean roughly a 50% chance once the bookmaker's margin is removed. During a match a goal changes these probabilities a lot, a red card moderately, and a yellow card or substitution only slightly.

Since odds are supposed to react to public information, a line that moves *before* any visible event, or moves much *more* than the event justifies, deserves a closer look. Integrity analysts look for exactly this kind of unexplained movement. Line Watch tries to answer one question: *at this point in the match, does the market's behaviour match what is happening on the pitch, and if not, how large is the gap?* We call this gap the **betting-line divergence**. In streaming/divergence_job.py it is defined for each match and five-minute window as:

```
divergence_score = actual_move - expected_move
```

actual_move is how much the implied probabilities really moved in the window and expected_move is how much they should have moved given its events. A value near zero or below means the events explain the movement; above 0.05 the window is flagged. The system follows Track D of the course guidelines (Kafka, Spark Streaming, Hive/HDFS) with a Streamlit dashboard on top.

2. Problem Statement

Existing problem. Betting markets produce continuously changing odds, while match events provide the information that should explain many of those changes; the two arrive as separate, unaligned streams. The challenge is to find windows where odds changed significantly but the events do not adequately explain it.

Proposed problem. Build a big data pipeline that replays odds and event streams, compares actual and expected movement in time windows, and flags **potentially suspicious unexplained movement**. The system is a screening tool only: a flagged window means the odds moved more than our rule-based model expects, not that match fixing or insider activity took place.

3. Objectives

1. Collect real betting odds (football-data.co.uk, Premier League 2023/24) and real event timelines (API-Football).
2. Generate minute-level odds ticks (minutes 0 to 90) from opening and closing odds using an event-weighted interpolation model.
3. Stream odds and events through two Kafka topics, odds-ticks and match-events, keyed by a shared match_id.
4. Process both streams with Spark Structured Streaming in five-minute windows: convert odds to normalised implied probabilities, compute actual and expected movement, the divergence score and a flag.
5. Store raw and scored data as Parquet on HDFS and expose them through Hive external tables.
6. Visualise odds, events and divergence with Streamlit and Plotly.
7. Validate the detector with unit tests and a clearly labelled synthetic leak scenario.

4. Dataset and Data Sources

4.1 Betting odds (real data)

fetch_odds_data.py downloads the free CSV football-data.co.uk/mmz4281/2324/E0.csv (Premier League 2023/24, one row per match) to data/raw/E0_2324.csv; odds_producer.py uses the Bet365 columns in Table 1. Only pre-match and closing lines are available, so the odds in between are modelled (Section 8), a stated assumption of the project.

Table 1. Input data fields used by the project

Source	Field(s)	Use in the project
E0_2324.csv	HomeTeam, AwayTeam, Date	Select the match row; fallback match_id
E0_2324.csv	B365H, B365D, B365A	Bet365 opening odds (home, draw, away)
E0_2324.csv	B365CH, B365CD, B365CA	Bet365 closing odds (home, draw, away)
API-Football	time.elapsed, time.extra	Event minute = elapsed + extra, clipped to 90
API-Football	type, detail	Mapped to Goal, Penalty Missed, Red/Yellow Card, Substitution
API-Football	team.name, player.name	Shown on the dashboard

4.2 Match events (real data)

events_producer.py calls API-Football v3 GET /fixtures/events?fixture=<id> (free tier, ~100 requests/day). Raw types are normalised: goal becomes Goal, or Penalty Missed if the detail says "missed"; card becomes Red Card (including a second yellow) or Yellow Card; subst becomes Substitution; anything else, such as VAR entries, is dropped because it duplicates an event already recorded. Stoppage-time events are clipped to minute 90, and results are cached locally. For Burnley 0-3 Manchester City (fixture 1035037, 11 August 2023) the API returns 15 raw events, which normalise to 14: 3 goals, 1 red card and 10 substitutions.

4.3 Synthetic validation data

producers/synthetic_leak_demo.py publishes one hand-built case with match_id = "SYNTH-LEAK-DEMO", a non-numeric id so it cannot be confused with a real fixture. We added it because a detector that never fires looks the same as a broken one. Home odds stay at 3.0 until minute 34 and crash to 1.8 in minutes 35 to 39 with no event nearby; the goal that would explain this comes only at minute 50, deliberately late, as if the market already knew. Window 35 should be flagged and window 50

should not. In summary, the odds CSV and API events are **real data**, the minute-level odds of real matches are **modelled** from real values, and SYNTH-LEAK-DEMO (plus the default sample_match_events.json) is **synthetic** and used only for validation.

5. Tools and Technologies

Table 2. Technologies used (versions from docker-compose.yml and requirements.txt)

Tool / technology	Version	Role in project
Python	3.12 (referenced in requirements.txt)	Producers, API access, odds model, dashboard
Apache Kafka, ZooKeeper; kafka-python-ng	Confluent cp-kafka / cp-zookeeper 7.6.1; client 2.2.2	Two-topic streaming ingestion
Apache Spark (PySpark)	3.1.1 cluster (bde2020), spark-sql-kafka-0-10_2.12:3.1.1; PySpark 3.5.1 for local tests	Structured Streaming job
Hadoop HDFS	3.2.1 (bde2020 namenode / datanode)	Parquet storage
Apache Hive	2.3.2 (bde2020/hive postgresql-metastore)	SQL layer (external tables)
PostgreSQL	bde2020/hive-metastore-postgresql:2.3.0	Hive metastore database
pandas, requests, python-dotenv, pytest	2.2.2, 2.32.3, 1.0.1, 8.3.2	CSV, HTTP, .env config, unit tests
Streamlit, Plotly	1.38.0, 5.24.1	Dashboard and charts
Docker Compose	Not specified in repository	Runs the 9-container stack

6. Tool-by-Tool Justification

Python handles everything before and after the big data stack: downloading odds, calling API-Football, normalising events, the interpolation model, the producers and the dashboard's HDFS reader.

Kafka decouples the two independently timed feeds from the consumer: odds arrive every match-minute, events only when something happens, and the Spark job can start before or after the producers and still read everything from the earliest offset. Keying every message by match_id lets Spark join the two topics per match.

Spark Structured Streaming does what no other tool in the stack can: reading two Kafka topics, windowing, aggregating, joining and scoring them.

HDFS is the persistent layer between processing and serving. The job writes Parquet under hdfs://namenode:8020/data/line_watch/, a columnar format that both Hive and pandas read directly. **Hive** gives SQL access through three external tables created by streaming/hive_tables.sql (metadata only; data is never copied). **PostgreSQL** is used only as the Hive metastore database (configured in hadoop-hive.env), not for application data.

Streamlit gave us a web dashboard in one Python file (dashboard/app.py), and **Plotly** draws the odds lines, event markers and divergence bars. **Docker Compose** starts all nine services (Kafka, ZooKeeper, HDFS, Hive, its PostgreSQL metastore and Spark) with one command.

7. System Architecture

Figure 1 shows the architecture as implemented. The Spark job runs three queries: two copy parsed Kafka rows into raw Parquet tables, and a heartbeat-driven third one recomputes the scores from them (dotted arrows).

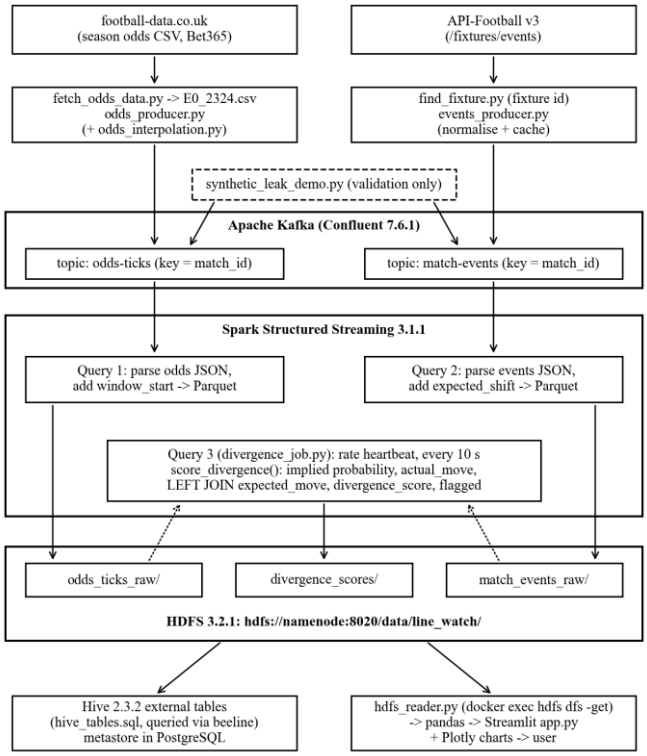


Figure 1. Overall architecture of the Line Watch system

8. Project Workflow and Data Flow

Stage 1: Data acquisition. Running fetch_odds_data.py for division E0 and season 2324 saves the season CSV as data/raw/E0_2324.csv.

Stage 2: Fixture identification. find_fixture.py --team Burnley --date 2023-08-11 returns the API-Football fixture id (1035037). A canonical match_id is needed because the sources spell team names differently ("Man City" and "Manchester City"). Otherwise the topics would carry different keys and the join would silently match nothing, so both producers get the same --match-id.

Stage 3: Odds interpolation. interpolate_odds() turns the opening odds, closing odds and events into 91 ticks. It works in de-vigged probability space, since probability is what events change. A weight *w(m)* between 0 and 1 says how much of the open-to-close move has happened by minute *m*:

```
w(m) = 0.3 * (m / 90) + 0.7 * E(m) / E(90)
E(m) = sum_e[ weight_e * sigmoid(1.5 * (m - minute_e)) ] / sum_e weight_e
p(m) = (1 - w) * p_open + w * p_close,    odds(m) = 1 / (overround(m) * p(m))
```

The 0.3 term is background drift; the sigmoids create steps at event minutes, weighted by EVENT_WEIGHTS (Table 3). The first and last ticks equal the real opening and closing odds.

Stages 4 and 5: Event normalisation and Kafka ingestion. Events are normalised as in Section 4.2. Both producers send JSON keyed by match_id: odds-ticks carries {match_id, minute, home_odds, draw_odds, away_odds, produced_at} and match-events carries {match_id, minute, type, team, player, produced_at}.

Stage 6: Spark processing. The job reads both topics from the earliest offset, parses them with from_json, and adds window_start (and expected_shift for events). Every 10 seconds a rate-source heartbeat re-reads the raw tables and runs score_divergence() (Section 9).

Stages 7 to 9: Storage, Hive and dashboard. Raw rows are appended to odds_ticks_raw and match_events_raw and scores overwrite divergence_scores under /data/line_watch/; hive_tables.sql creates the external tables; hdfs_reader.py copies tables out with hdfs dfs -get and docker cp for app.py to plot.

9. Analytical Methodology

The scoring logic is score_divergence() in divergence_job.py, kept apart from the Kafka and HDFS code so it can be unit tested (Figure 2).

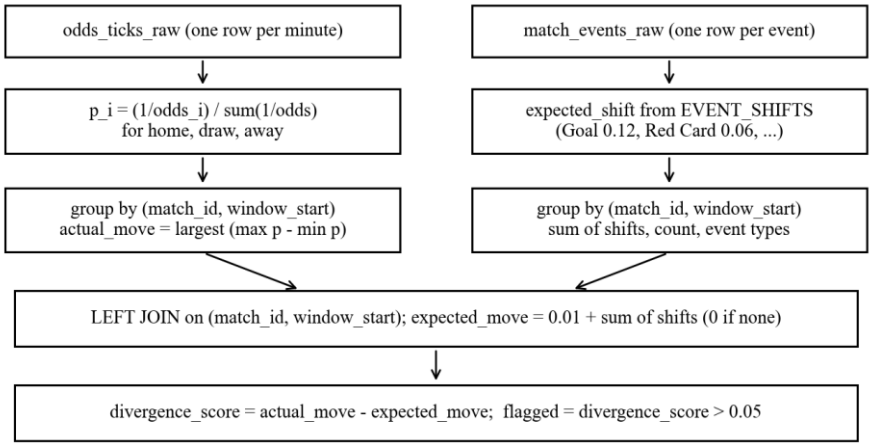


Figure 2. Divergence calculation workflow inside score_divergence()

9.1 Implied probability. Dividing each inverse odd by their sum removes the bookmaker's margin, so a change in margin alone is not mistaken for market movement:

```
p_home = (1/home_odds) / (1/home_odds + 1/draw_odds + 1/away_odds)    (same for draw, away)
```

9.2 Five-minute window. window_start = floor(minute / 5) * 5, so minutes 35 to 39 form window 35. A match gives 19 windows (0 to 90).

9.3 Actual movement is the largest probability range among the three outcomes in the window:

```
actual_move = max( range(p_home), range(p_draw), range(p_away) ),  range = max - min
```

9.4 Expected movement is expected_move = 0.01 + sum of expected_shift over the window's events. The 0.01 allows for a market that is never perfectly still. The priors (Table 3) are deliberately **not** the producer's EVENT_WEIGHTS; reusing the table that generated the odds would make divergence near zero by construction.

Table 3. Event weights used to shape the odds vs. event shifts used as expected movement

Event type	EVENT_WEIGHTS (odds_interpolation.py, relative)	EVENT_SHIFTS (divergence_job.py, probability)
Goal	1.0	0.12
Red Card	0.5	0.06
Penalty Missed	0.3	0.04
Yellow Card	0.15	0.02
Substitution	0.05	0.01
Any other type	0.1	0.01

9.5 Divergence and 9.6 flagging. divergence_score = actual_move - expected_move, and flagged is true when the score is above 0.05 (DIVERGENCE_THRESHOLD), false otherwise. Actual and expected movement are combined with a **left join** on (match_id, window_start), so a window where

odds moved with no event is kept (expected 0.01); an inner join would silently drop this key case. The detector is entirely **rule-based**, with fixed formulas, priors and threshold; no model is trained and no machine learning is used.

10. Results

Synthetic leak case. Table 4 shows the scores for SYNTH-LEAK-DEMO. Windows 35 and 50 match streaming/README.md (cluster run); we recomputed all 19 windows offline with the same formulas and got the same values.

Table 4. Divergence results for the synthetic leak case (19 windows, 1 flagged)

Window	Events	Actual move	Expected move	Divergence	Flagged
0 to 30	None	0.000	0.010	-0.010	No
35	None	0.194	0.010	+0.184	Yes
40, 45	None	0.000	0.010	-0.010	No
50	Goal	0.003	0.130	-0.127	No
55 to 85	None	0.003	0.010	-0.007	No
90	None	0.000	0.010	-0.010	No

In window 35 the home probability rises from 0.324 (odds 3.0/3.4/2.5) to 0.519 (1.8/3.6/4.2) with nothing to explain it, so it is flagged. Window 50 has a Goal (expected 0.13), but the market barely moves because it was already priced in, so it is correctly not flagged. The tool scores moves against the information public at the time, not just their size (Figure 3).

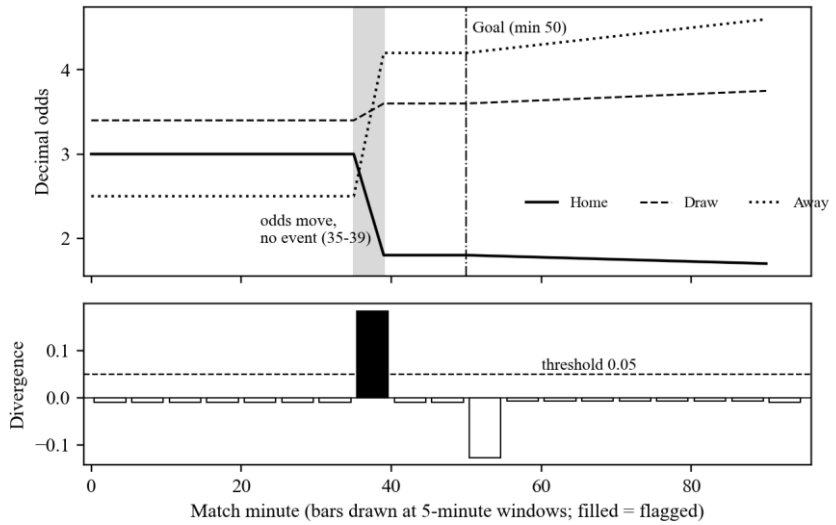


Figure 3. Synthetic leak case: odds and goal marker (top), divergence per window (bottom); recomputed from build_ticks() with the project's scoring formulas

Real matches. For Burnley 0-3 Manchester City (fixture 1035037) the producers published 91 odds ticks and 14 events, with the first and last ticks equal to the real lines. The job produced 19 scored windows, checked through Parquet and Hive. **None of the 19 windows were flagged.** Burnley's win probability moved only from about 11.8% (odds 8.0) to 10.6% over the whole match. Tottenham 2-0 Manchester United, with closer odds, also produced **zero flagged windows**. The main reason is that real-match odds are interpolated between two lines and are far smoother than a real in-play feed.

11. Analytical Validity

The real matches show the detector does not raise false alarms; the synthetic case shows that an unexplained move is flagged while a move with an appropriate event is not.

A flagged window indicates unexplained statistical movement according to the project's rule-based model; it is not evidence by itself of match fixing or insider activity. Our results show that the pipeline and scoring behave as designed, not that the system detects real-world fixing (Table 5).

Table 5. Scope summary

Category	Items
Implemented	Data download, event normalisation, odds interpolation, three Kafka producers, Spark streaming job, Parquet on HDFS, Hive external tables, Streamlit dashboard
Tested	Interpolation, event normalisation on a real API response, synthetic ticks, divergence scoring including the left-join case (17 automated tests)
Demonstrated	End-to-end runs (two real matches, synthetic case) checked via HDFS, Hive and the dashboard, per the READMEs
Assumed / modelled	Minute-level odds for real matches, event priors, 0.01 drift, 0.05 threshold
Future work only	Real tick-level odds, several bookmakers, statistical or ML models, dynamic thresholds, cloud deployment

12. Implementation and Integration

The tools are connected by real data hand-offs (Table 6). Spark writes plain Parquet to fixed paths instead of using Hive's catalog, avoiding a Spark/Hive 2.3.2 client version clash, and each trigger recomputes the windowed join from the raw tables, which is cheap at 100 to 200 rows per match.

Table 6. Data entering and leaving each stage

Stage	Data in	Data out
Sources to Python	E0.csv over HTTP; API-Football JSON	E0_2324.csv; cached event JSON
Python to Kafka	91 interpolated ticks; normalised events	JSON messages keyed by match_id on two topics
Kafka to Spark	Topic records (binary key/value)	Typed DataFrames with window_start, expected_shift
Spark to HDFS	Parsed streams; scored batch result	Parquet: odds_ticks_raw, match_events_raw, divergence_scores
HDFS to Hive	Parquet at fixed paths	SQL tables queried through beeline
HDFS to Streamlit	Parquet via hdfs dfs -get and docker cp	pandas DataFrames, Plotly charts, tables

13. Progress Check-ins

Check-in 1: Working ingestion. The producer commits (9247200, 66e4060) are dated 14 September 2026. producers/README.md records 14 real events and 91 odds ticks for fixture 1035037, with both topics carrying key 1035037 (checked with kafka-console-consumer). The repository does not contain a dated progress-check-in record for this stage; the implementation itself demonstrates the completed ingestion stage.

Check-in 2: Draft analytical results. The streaming job (commit 93d53b0) and the synthetic case (commit 8675da6) are dated 21 September 2026. streaming/README.md records 19 unflagged windows for fixture 1035037 and the window-35 flag (+0.184) for the synthetic case. The repository does not contain a dated progress-check-in record for this stage; the implementation itself demonstrates the completed analysis stage.

14. Demo Workflow

A ten-minute demo focused on evidence of real execution:

- **0:00 to 2:00** Problem statement, architecture (Figure 1) and the divergence formula. **2:00 to 4:00** docker compose up -d; containers, HDFS UI, Spark UI.
- **4:00 to 5:30** Run the three producers; show messages with kafka-console-consumer. **5:30 to 7:00** spark-submit the job; show "scored N windows, M flagged".
- **7:00 to 8:00** HDFS folders and a beeline query on divergence_scores. **8:00 to 9:30** Streamlit dashboard: real matches and SYNTH-LEAK-DEMO.
- **9:30 to 10:00** The flagged window, the unflagged goal window, and the interpolation limitation.

15. Challenges Encountered

Table 7 lists challenges recorded in the code comments, READMEs and commit messages.

Table 7. Challenges: problem, impact and solution

Problem	Impact	Solution / workaround
No free tick-level odds	Nothing realistic to stream in-play	Event-weighted interpolation in probability space
Team names differ between sources	Different keys; Spark join finds nothing	Explicit shared --match-id (API-Football fixture id)
API limit of ~100 requests/day	Re-runs would exhaust the quota	Cache events in data/interim/events/
Version problems: Kafka image, kafka-python and Spark 3.1.1 with Python 3.12	Ingestion and local tests broke	Confluent 7.6.1 images, kafka-python-ng; PySpark 3.5.1 for local tests only
Stream-stream outer joins in Spark 3.1	Event-less windows at risk	Recompute a batch left join on every trigger
ProcessingTime trigger fires only on new offsets; early triggers before tables exist	Score table froze after a replay; early failures	rate-source heartbeat every 10 s; skip the batch if tables are not ready
HiveServer2 2.3.2 SASL vs. Python clients; WebHDFS redirect to unpublished datanode	Dashboard could not read via Hive or WebHDFS	hdfs dfs -get in the namenode container, docker cp, pandas

16. Limitations

1. **Interpolated odds.** Real in-play volatility is not captured, the main reason real matches show no flags. *Improvement:* in-play odds snapshots (the plan names the Apify football-odds-historical actor).
2. **Rule-based detector with hand-set priors.** EVENT_SHIFTS ignores score and minute. *Improvement:* priors conditioned on game state or learned from history.
3. **Fixed 0.05 threshold** for all matches. *Improvement:* volatility-scaled thresholds.
4. **Window boundaries.** A jump between minutes 34 and 35 falls between windows and is not counted. Found in our code review; not handled. *Improvement:* sliding windows.
5. **Default event file.** Without --events, odds_producer.py shapes the curve with the synthetic sample_match_events.json, and the README command does not pass it. For real matches --events data/interim/events/<id>.json should be given.

17. Future Work

Proposed, not implemented: a real in-play odds API and several bookmakers; expected movement learned from historical matches, statistical anomaly detection (for example per-window z-scores) or

machine-learning models; dynamic thresholds; incremental stateful Spark processing with watermarks; cloud deployment instead of the current single-host Docker setup; and a proper query service instead of the docker exec/docker cp dashboard workaround.

18. Individual Contribution Breakdown

The team members are Jen Jose Jeeson (2023BCS0017), Rupsika Thipparthi (2023BCS0197) and Praneetha Pillai (2023BCS0207). The Git history records all 14 commits (8 content commits and the merges of PR #1 to #6) under the GitHub account jenjose72 (Jen Jose Jeeson), so it does not show which member built which module. Table 8 is inferred from the repository organisation; the team must fill in and verify the member column before submission.

Table 8. Role-based contribution breakdown (member assignment to be confirmed by the team)

Role	Files / modules	Contribution	Team member
A: Ingestion	fetch_odds_data.py, find_fixture.py, odds_interpolation.py, odds_producer.py, events_producer.py, synthetic_leak_demo.py	Data pulls, interpolation model, Kafka producers, validation case	To be confirmed
B: Analytics	divergence_job.py, hive_tables.sql	Streaming job, windowed join, scoring, HDFS/Hive output	To be confirmed
C: Serving and reporting	hdfs_reader.py, app.py, READMEs	Dashboard, HDFS access, documentation, demo and report	To be confirmed

19. Reproducibility and Setup

Requirements are Docker with Compose, Python with requirements.txt, and an API-Football key. The commands are taken from the repository READMEs:

```
git clone https://github.com/jenjose72/betting-line-divergence.git
cd betting-line-divergence && cp .env.example .env    # set API_FOOTBALL_KEY
docker compose up -d
python -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
python -m producers.fetch_odds_data --division E0 --season 2324
python -m producers.find_fixture --team Burnley --date 2023-08-11
python -m producers.events_producer --fixture-id 1035037 \
  --match-id 1035037 --speed 0
python -m producers.odds_producer --home Burnley --away "Man City" \
  --match-id 1035037 --speed 0
python -m producers.synthetic_leak_demo --speed 0
docker exec linewatch-spark-master /spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.1.1 \
  /workspace/streaming/divergence_job.py --bootstrap-servers kafka:29092
cat streaming/hive_tables.sql | docker exec -i linewatch-hive-server \
  beeline -u jdbc:hive2://localhost:10000
streamlit run dashboard/app.py
python -m pytest
```

20. Testing

The repository has 17 unit tests in four files (Table 9). We ran the suite on 28 September 2026 on Windows 11 and all 17 passed. The 15 producer tests needed PYTHONUTF8=1 on Python 3.13, because the API fixture (containing "Kovačić") is opened without an explicit encoding and Windows defaults to cp1252. The two Spark tests needed Python 3.11 with JDK 21 (they failed on JDK 25 / Python 3.13).

Table 9. Automated test cases and results

Test file	Tests	What is validated	Result
producers/tests/ test_odds_interpolation.py	5	91 ticks; first = opening and last = closing odds; works with no events; bigger move around a goal than in a quiet period	5 passed
producers/tests/ test_events_normalization.py	5	Real response for 1035037: 3 goals, 1 red, 10 subs; VAR dropped; 90+4 clipped to 90; sorted; first goal Haaland, minute 4	5 passed
producers/tests/ test_synthetic_leak_demo.py	5	91 ticks; flat before the leak; drop above 1.0 in minutes 35 to 39; holds; ends at final close	5 passed
streaming/tests/ test_divergence_scoring.py	2	Same move flagged without an event, not flagged with a Goal; event-less window kept (left join)	2 passed

21. Evaluation Rubric Mapping

Table 10. Course evaluation criteria and evidence in the project

Evaluation criterion	Evidence in project
Proposal & Planning	PROJECT_PLAN.md: problem, data, Track D tools, schedule, risks, outcome
Implementation & Integration	Python, Kafka (2 topics), Spark Streaming, HDFS, Hive, Streamlit on real data (Sections 8, 12)
Analytical Quality	Actual vs. expected movement, independent priors, left join, synthetic true positive, real true negatives (Sections 9 to 11)
Demo	Ten-minute live run of the full pipeline (Section 14)
Documentation & Report	READMEs, Figure 1, setup commands (Section 19), 17 tests
Presentation & Q&A	Documented design decisions and stated limitations (Sections 12, 15, 16)

22. Conclusion

Line Watch compares football betting-line movement with the events that should explain it, using a Kafka, Spark Structured Streaming, HDFS/Hive and Streamlit pipeline. The analysis is rule-based, comparing actual movement in implied probability with expected movement from independent event priors. The synthetic leak was flagged in the expected window and its late goal left unflagged, while two real matches produced no false alarms. The main limitation is interpolated real-match odds; real in-play data and a data-driven expected model are the next steps.

23. References

[1] football-data.co.uk, football results and betting odds data (Premier League 2023/24, mmz4281/2324/E0.csv). https://www.football-data.co.uk/

[2] API-SPORTS, API-Football v3 (endpoints /fixtures/events, /fixtures, /teams). https://api-sports.io

[3] Apache Software Foundation, Apache Kafka. https://kafka.apache.org/

[4] Confluent, Docker images confluentinc/cp-kafka and cp-zookeeper 7.6.1. https://hub.docker.com/u/confluentinc

[5] Apache Software Foundation, Apache Spark 3.1.1 Structured Streaming Programming Guide. https://spark.apache.org/docs/3.1.1/structured-streaming-programming-guide.html

[6] Apache Software Foundation, Apache Hadoop. https://hadoop.apache.org/

[7] Apache Software Foundation, Apache Hive. https://hive.apache.org/

[8] Big Data Europe, Docker images for Hadoop, Hive and Spark (bde2020). https://github.com/big-data-europe

[9] Streamlit. https://streamlit.io/

[10] Plotly, Plotly Python graphing library. https://plotly.com/python/

[11] kafka-python-ng, Python client for Apache Kafka. https://pypi.org/project/kafka-python-ng/

[12] CSE412 Big Data & Large-Scale Computing, Applied Project Guidelines (course document).

[13] J. J. Jeeson, R. Thipparthi and P. Pillai, Line Watch project repository. https://github.com/jenjose72/betting-line-divergence